

Caractéristiques des aéronefs et CRC

Javions – étape 2

1. Introduction

Cette seconde étape du projet Javions a deux buts :

1. définir un ensemble de classes représentant différentes caractéristiques fixes des aéronefs visibles sur la carte, obtenues d'une base de données,
2. définir une classe calculant ce que l'on appelle un CRC, qui permet de vérifier – dans une certaine mesure – la validité des messages ADS-B reçus.

Avant de commencer à travailler à cette étape, lisez le guide [Sauvegarder son travail](#). Il vous donnera des conseils importants concernant la sauvegarde de votre projet au cours du semestre.

2. Concepts

2.1. Caractéristiques fixes des aéronefs

Parmi les différentes caractéristiques d'un aéronef, certaines sont fixes dans le sens où elles ne varient pas – ou presque pas – durant sa vie. Par exemple, tout aéronef possède un numéro d'immatriculation, visible sur son fuselage, qui ne change généralement pas.

Étant donné que ces caractéristiques sont fixes, elles ne sont pas transmises dans les messages envoyés par les aéronefs. Pour les obtenir, il est donc nécessaire d'utiliser d'autres sources de données. Dans le cadre de ce projet, nous utiliserons une base de données publique mise à disposition par l'entreprise [mictronics](#).

Cette base de données contient les attributs de plus de 400 000 aéronefs, identifiés par ce que l'on appelle leur **adresse OACI** (*ICAO address*). Cette adresse est un nombre unique attribué à tout aéronef par l'organisation de l'aviation civile internationale (OACI, ou *ICAO* en anglais, pour *international civil aviation organization*). L'adresse OACI fait 24 bits, et est généralement écrite en hexadécimal (base 16), avec 6 chiffres exactement. Pour cette raison, elle est parfois nommée simplement *hex*.

Tous les messages ADS-B envoyés par un aéronef contiennent son adresse OACI, ce qui permet d'une part de regrouper tous les messages émis par un aéronef donné, et d'autre part de trouver les informations relatives à l'émetteur dans la base de données.

La version librement téléchargeable de la base de données mictronics fournit, entre autres, les informations suivantes sur les aéronefs :

- leur **numéro d'immatriculation** (*registration number*),
- leur **indicateur de type** (*type designator*), défini par le [document 8643 de l'OACI](#),
- leur **description** (*description*), un code de trois lettres donnant le type de l'aéronef, son nombre de moteurs et son type de propulsion,

- leur **modèle** (*model*), donnant généralement le nom du fabricant et celui du modèle spécifique de l'aéronef,
- leur **catégorie de turbulence de sillage** (*wake turbulence category*, souvent abrégée *WTC*), qui donne une indication de l'importance des turbulences produites dans le sillage de l'aéronef.

La table suivante donne un exemple d'une entrée de la base de données, qui décrit un avion de la compagnie Swiss :

Caractéristique	Valeur
Adresse OACI	4B1814
Immatriculation	HB-JDC
Indicateur de type	A20N
Description	L2J
Modèle	AIRBUS A-320neo
Catégorie de turbulence de sillage	M (<i>medium</i>)

Le préfixe utilisé pour l'immatriculation (HB) nous permet de savoir que l'avion a été immatriculé en Suisse. Sa description (L2J) nous apprend qu'il s'agit d'un avion qui se pose sur terre – L signifiant *land* –, qu'il possède 2 moteurs, et que ces moteurs sont à réaction – J signifiant *jet propulsion*.

Une photo de cet avion, obtenue du site [Planespotting.net](https://www.planespotting.net), est visible ci-dessous. En cliquant dessus, on accède à une version à plus haute résolution, qui permet de vérifier la plupart de ces caractéristiques.



Figure 1 : L'avion immatriculé HB-JDC (photo ©Oscar Fernandez)

Même si la base de données micronics est conséquente, elle ne contient pas la totalité des aéronefs existants, et même lorsqu'elle contient des informations au sujet d'un aéronef, il est possible qu'elles ne soient que partielles. Dès lors, il arrive qu'aucune entrée n'existe pour une adresse OACI valide, ou alors que certains des attributs mentionnés plus haut soient vides.

2.2. Format des données

La base de données micronics est [disponible au téléchargement](#) dans plusieurs formats, mais aucun d'entre eux ne correspond vraiment à nos besoins. Nous l'avons donc réorganisée et nous vous l'avons fournie sous la forme d'une archive Zip nommée `aircraft.zip`, incluse dans le squelette.

Attention toutefois : contrairement à d'autres archives Zip fournies dans le cadre de ce projet, le contenu de celle-ci n'est *pas* destiné à être extrait avant d'être utilisé. Au lieu de cela, le code que vous écrirez dans le cadre de cette étape accédera directement au contenu de cette archive.

L'archive `aircraft.zip` contient 256 fichiers, dont le nom est composé de deux caractères hexadécimaux (de 0 à 9 ou de A à F) suivis de l'extension `.csv`. Comme cette extension l'indique, ces fichiers sont au format CSV, pour *comma-separated values*. Ce format permet de représenter, sous forme textuelle assez compacte, les lignes d'une table : chaque ligne de la table est représentée par une ligne du fichier, et les colonnes sont séparées par une virgule (*comma* en anglais).

Par exemple, la ligne 293 du fichier nommé `14.csv` de l'archive est :

```
4B1814,HB-JDC,A20N,AIRBUS A-320neo,L2J,M
```

qui correspond à l'aéronef dont l'adresse OACI est 4B1814, utilisé comme exemple plus haut. De gauche à droite, les colonnes contiennent : l'adresse OACI, le numéro d'immatriculation, l'indicateur de type, le modèle, la description et la catégorie de turbulence de sillage.

Il faut noter que les fichiers CSV sont souvent dotés d'une ligne d'en-tête dont le but est de nommer les différentes colonnes. Pour les fichiers de la base de données, cette ligne d'en-tête pourrait ressembler à ceci :

```
ICAO,Registration,Designator,Model,Description,WTC
```

Attention toutefois, les fichiers de l'archive contenant la base de données **ne disposent pas** de ligne d'en-tête, dans un souci de simplicité et de gain de place.

Les 256 fichiers de l'archive contiennent chacun entre 1500 et 1900 aéronefs. Ceux-ci sont répartis dans les différents fichiers en fonction des deux derniers caractères de leur adresse OACI. Par exemple, l'aéronef dont l'adresse est 4B1814 se trouve dans le fichier nommé `14.csv`. De plus, les fichiers sont triés par adresse OACI, en ordre croissant.

Le fait que les données soient ainsi partitionnées en plusieurs fichiers et triées permet de trouver les données d'un aéronef dont on connaît l'adresse OACI de manière assez efficace. Il suffit en effet de :

1. ouvrir le fichier dont le nom correspond aux deux dernières lettres de l'adresse OACI recherchée,
2. le parcourir ligne à ligne jusqu'à trouver la première entrée dont l'adresse est plus grande ou égale à celle recherchée.

Si aucune ligne n'est trouvée, ou si l'adresse de celle trouvée ne correspond pas à l'adresse recherchée, alors l'aéronef ne figure pas dans la base. Sinon, la ligne trouvée contient ses informations.

2.3. Chaînes contraintes

Comme nous l'avons vu plus haut, la plupart des caractéristiques fixes des aéronefs peut être représentée par de simples chaînes de caractères.

Ces chaînes ne sont toutefois pas quelconques, mais obéissent à des formats bien particuliers, et nous dirons donc qu'elles sont contraintes. Par exemple, la chaîne représentant l'adresse OACI d'un avion fait exactement 6 caractères, et chacun d'entre

eux doit être soit un chiffre décimal (0 à 9), soit l'une des six premières lettres majuscules de l'alphabet latin (A à F).

Pour spécifier le format de ces chaînes, nous utiliserons ce que l'on appelle des **expressions régulières** (*regular expressions*), notées ER dans ce qui suit, et qui permettent de décrire la syntaxe de chaînes de caractères.

Une ER est une notation compacte pour décrire un ensemble, souvent infini, de chaînes de caractères – un peu comme une fonction mathématique est une notation compacte pour décrire un ensemble, souvent infini, de (couple de) valeurs. Les ER sont très utiles et donc souvent utilisées en informatique, même si ce n'est pas toujours à bon escient.¹

Les ER les plus simples sont les ER constantes, qui représentent un ensemble de chaîne qui ne contient qu'un seul élément. Par exemple, l'ER que nous noterons `pomme` – le cadre servant à la distinguer d'une chaîne normale – représente l'ensemble de chaîne de caractères qui ne contient qu'un seul élément, la chaîne pomme.

Deux ER peuvent être combinées au moyen d'un opérateur de choix, généralement noté par une barre verticale (`|`). Par exemple, l'ER `pomme|poire` représente l'ensemble de chaînes de caractères qui contient exactement deux éléments : la chaîne pomme et la chaîne poire. (La barre verticale est en rouge car nous utilisons cette couleur pour distinguer les opérateurs des parties constantes des ER dans ce qui suit.)

Une notation abrégée existe pour noter le choix entre plusieurs ER constituées d'un seul caractère, et consiste à entourer de crochets une séquence de caractères. Par exemple, l'ER `[GACT]` est équivalente à l'ER `G|A|C|T`. Entre les crochets, un tiret (`-`) désigne une plage de caractères, sauf s'il est placé juste avant le crochet fermant. Par exemple, l'ER `[A-D]` est équivalente à l'ER `A|B|C|D`, mais l'ER `[A-]` est équivalente à `A|`, car le tiret se trouve juste avant le crochet fermant et perd ainsi sa signification particulière.

En plus de l'opérateur de choix, plusieurs opérateurs existent pour désigner la répétition. Ainsi, en plaçant un entier entre accolades après une ER, on dénote sa répétition ce nombre de fois-là. Par exemple, l'ER `a{3}rgh` est équivalente à l'ER constante `aaargh`. En plaçant deux entiers séparés par une virgule entre les accolades, on dénote la répétition un nombre de fois compris entre le premier et le second entier (inclus). Ainsi, l'ER `a{0,2}rgh` est équivalente à l'ER `rgh|argh|aargh`.

De manière similaire, l'opérateur « plus » (`+`) placé après une ER dénote sa répétition un nombre quelconque, mais non nul, de fois. Ainsi, l'ER `a+rgh` représente l'ensemble infini de chaînes de caractères contenant, entre autres, les chaînes `argh`, `aargh`, `aaargh`, etc.

Il existe encore de nombreux autres opérateurs sur les ER, mais ceux décrits ci-dessus nous suffiront. Malheureusement, les opérateurs disponibles ne sont pas totalement standardisés, et il existe donc plusieurs versions d'ER dont les notations ne sont que partiellement compatibles. Dans ce projet, nous utiliserons les ER représentées par la classe `Pattern` de la bibliothèque Java, dont la syntaxe est décrite en détail dans [la documentation de la classe](#).

Dans cette notation, les ER décrivant les différents types de chaînes contraintes qui nous intéressent sont données dans la table ci-dessous. En cliquant sur l'une d'entre elles,

vous accéderez à sa représentation graphique, qui peut faciliter sa compréhension.

Attribut	Expression régulière	Exemple
Adresse OACI	<code>[0-9A-F]{6}</code>	4B1814
Immatriculation	<code>[A-Z0-9 .?/_+-]+</code>	HB-JDC
Indicateur de type	<code>[A-Z0-9]{2,4}</code>	A20N
Description	<code>[ABDGHLPRSTV-][0123468][EJPT-]</code>	L2J

2.4. Somme de contrôle

Les messages ADS-B envoyés par les aéronefs peuvent être corrompus lors de leur transmission. Cela se produit par exemple lorsque des messages envoyés par des aéronefs différents se chevauchent. Dès lors, il est nécessaire d'avoir à disposition un mécanisme permettant de vérifier, autant que possible, la validité des messages reçus. De tels mécanismes sont généralement basés sur l'ajout de redondance aux messages.

Par exemple, on pourrait imaginer que les aéronefs envoient toujours deux copies successives d'un message donné, et que le récepteur ignore les messages dont les deux copies ne sont pas identiques. Un tel mécanisme permet de détecter certaines erreurs de transmission, mais est coûteux puisqu'il double la quantité de données à transmettre.

En pratique on utilise donc un mécanisme moins coûteux, qui consiste à attacher à chaque message une petite quantité de données dont la valeur dépend du contenu du message. Ces données, qui ne servent qu'à augmenter la probabilité de détecter d'éventuelles erreurs de transmission, sont appelées **sommes de contrôle** (*checksums*). Malgré leur nom, ces sommes de contrôle ne sont généralement pas calculées au moyen de simples sommes.

L'exemple le plus simple de somme de contrôle est le contrôle de parité. Il consiste à ajouter à chaque message un unique **bit de parité** (*parity bit*), dont la valeur est déterminée par la parité du nombre de bits valant 1 dans le message : si ce nombre de bits est pair, alors le bit de parité vaut 1, sinon il vaut 0. Le récepteur d'un message peut alors compter le nombre de bits valant 1 dans le message qu'il a reçu et vérifier que cette valeur est conforme au bit de parité. Si c'est le cas, alors le message est considéré comme valide, sinon il est considéré comme invalide, et est ignoré.

Un bit de parité permet de détecter la transmission incorrecte d'un nombre impair de bits du message, mais il ne peut pas détecter les cas où un nombre pair de bits a été reçu de manière incorrecte.

Les messages ADS-B utilisent une somme de contrôle plus sophistiquée qu'un simple bit de parité, nommée **contrôle de redondance cyclique** (*cyclic redundancy check*), abrégée CRC et décrite ci-dessous. Le CRC attaché aux messages ADS-B fait 24 bits, mais des CRC d'autres tailles – en particulier 32 bits – sont fréquents en informatique.

2.4.1. Contrôle de redondance cyclique

Le calcul d'un CRC se fait au moyen d'une opération qui ressemble beaucoup à une division entre deux nombres², le reste de cette division constituant le CRC. Le numérateur de la division est le message, vu comme un nombre en base 2, tandis que son dénominateur, nommé **générateur** (*generator*), est fixe et propre à la variante de CRC

utilisée. Le générateur fait toujours un bit de plus que le CRC, et son bit de poids le plus fort vaut toujours 1.

Pour illustrer la technique de calcul, voyons comment calculer le CRC4 – c.-à-d. de 4 bits – du message 11111100111_2 en utilisant le générateur 11001_2 . La première opération à effectuer est d'augmenter le message en lui ajoutant 4 bits – autant que le CRC à calculer – valant 0 à la fin. Ensuite, on procède presque comme pour une division normale effectuée sur papier, mais en travaillant en base 2 et en utilisant le « ou exclusif » bit à bit – indiqué plus bas par l'opérateur $\wedge$ – plutôt que la soustraction.

Ce calcul est présenté ci-dessous, les quatre bits ajoutés à la fin du message ayant été mis en évidence en bleu pour faciliter la compréhension.

```

111111001110000
^ 110010000000000
-----
001101001110000
^ 110010000000000
-----
0001101110000
^ 11001000000
-----
0001010000
^ 1100100
-----
0110100
^ 110010
-----
000110 = CRC

```

Le reste de cette division (un peu spéciale) vaut 0110_2 , et c'est donc la valeur du CRC4. Notez que le quotient de la division n'est pas indiqué ci-dessus, car il ne joue aucun rôle dans le calcul du CRC et est donc généralement ignoré.

2.4.2. Vérification de CRC

Pour vérifier le CRC d'un message, la technique évidente consiste à le calculer comme illustré ci-dessus, et vérifier que le CRC calculé est égal à celui attaché au message.

Une technique plus simple est généralement utilisée en pratique. Elle consiste à concaténer le message et le CRC, calculer le CRC de cette combinaison, et vérifier qu'il vaut 0. Cette technique est illustrée ci-dessous pour valider le message plus haut et son CRC.

Pour faciliter la compréhension de la première ligne, le CRC y a été mis en évidence en rose, les quatre bits ajoutés à la fin du message en bleu.

```

1111110011101100000
^ 1100100000000000000
-----

```

```

0011010011101100000
^ 11001000000000000
-----
00011011101100000
^ 110010000000000
-----
000101011000000
^ 11001000000
-----
01100100000
^ 1100100000
-----
00000000000 OK

```

2.4.3. Algorithme de base (bit par bit)

La technique de calcul de CRC illustrée plus haut peut être transformée en un algorithme basique donné ci-dessous en pseudo-code. Dans cet algorithme, N est la taille du CRC en bits, et G est le générateur, dont le bit d'index N vaut 1. (Pour mémoire, les bits sont numérotés de droite à gauche, le bit le plus à droite ayant l'index 0 et le poids le plus faible.)

```

1: crc = 0
2: pour chaque bit b du message augmenté, de gauche à droite:
3:   crc = (crc << 1) | b
4:   si crc[N] == 1:
5:     crc = crc ^ G
6: crc = N bits de poids faible de crc

```

Les opérateurs $<<$, $|$ et $^$ sont les mêmes que ceux de Java, et l'expression $\text{crc}[N]$ représente le bit d'index N de crc .

Le fonctionnement de cet algorithme lors du calcul du CRC4 du message 11111100111 en utilisant le générateur 11001 est présenté dans la table ci-dessous. Les colonnes donnent la valeur des variables b et crc . La valeur de cette dernière est donnée après l'exécution de la ligne 3 et après celle de la ligne 5. Pour faciliter la lecture, le bit d'index 4 de crc au moment du test est souligné dans la seconde colonne, tandis que la valeur de crc après l'exécution de la ligne 5 est grisé dans la troisième colonne lorsque ce bit vaut 0 et donc que la ligne 5 n'est pas exécutée.

b	crc @3	crc @5
1	<u>0</u> 0001	00001
1	<u>0</u> 0011	00011
1	<u>0</u> 0111	00111
1	<u>0</u> 1111	01111
1	<u>1</u> 1111	00110
1	<u>0</u> 1101	01101
0	<u>1</u> 1010	00011
0	<u>0</u> 0110	00110

b	crc @3	crc @5
1	<u>0</u> 1101	01101
1	<u>1</u> 1011	00010
1	<u>0</u> 0101	00101
0	<u>0</u> 1010	01010
0	<u>1</u> 0100	01101
0	<u>1</u> 1010	00011
0	<u>0</u> 0110	00110

Cet algorithme de base peut être modifié de plusieurs manières afin d'obtenir progressivement une version plus optimisée. Tout d'abord, il est possible d'inverser la mise à jour de la variable `crc` et le test de son bit de poids fort, pour obtenir :

```
crc = 0
pour chaque bit b du message augmenté, de gauche à droite:
    si crc[N - 1] == 1:
        crc = ((crc << 1) | b) ^ G'
    sinon:
        crc = (crc << 1) | b
crc = N bits de poids faible de crc
```

Cette seconde version semble un peu plus complexe mais elle offre l'avantage de permettre le calcul d'un CRC d'une taille donnée avec une variable `crc` de même taille. Par exemple, un CRC32 peut être calculé avec un entier Java de 32 bits (type `int`), alors que cela n'était pas possible avec la version précédente. De même, le bit de poids fort du générateur, qui vaut toujours 1, n'a pas besoin d'être stocké, et le générateur peut donc être représenté au moyen du même nombre de bits que le CRC. C'est la raison pour laquelle cette seconde version de l'algorithme utilise `G'`, qui contient les `N` bits de poids faible de `G`, plutôt que `G` lui-même.

Une autre amélioration possible consiste à utiliser une table à deux entrées (0 et `G'`) indexée par le bit d'index `N-1` de la variable `crc` pour la mettre à jour plutôt qu'un test :

```
table = [ 0, G' ]
crc = 0
pour chaque bit b du message augmenté, de gauche à droite:
    crc = ((crc << 1) | b) ^ table[crc[N - 1]]
crc = N bits de poids faible de crc
```

Le fait que la table contienne 0 à l'index 0 implique que le « ou exclusif » n'a aucun effet si le bit d'index `N-1` de `crc` vaut 0.

2.4.4. Algorithme optimisé (octet par octet)

Les différentes versions de l'algorithme présentées à la section précédente travaillent toutes bit par bit. Or on peut transformer la dernière version en un algorithme travaillant octet par octet en utilisant une table contenant 256 (2^8) éléments plutôt que 2 (2^1) :

```
table = [ 0, ... ]
```



```
crc = 0
pour chaque octet o du message augmenté, de gauche à droite:
    crc = ((crc << 8) | o) ^ table[crc[N-1 ... N-8]]
crc = N bits de poids faible de crc
```

L'expression `crc[N-1 ... N-8]` représente l'octet de poids fort de `crc`.

Le problème est de savoir comment calculer la table. Or cela peut se faire simplement en appliquant l'algorithme bit à bit à tous les messages formés d'un seul octet. Donc si on note `crc_b` l'algorithme de la section précédente, la table peut être calculée ainsi :

```
table = [ crc_b([0]), crc_b([1]), ..., crc_b([255]) ]
```

Il est encore possible d'apporter une dernière amélioration à cet algorithme, qui évite de devoir augmenter le message en lui ajoutant des 0 à la fin. C'est cette version qui est généralement utilisée en pratique. Toutefois, comme elle est relativement difficile à comprendre, nous nous contenterons de la version ci-dessus pour ce projet.

2.4.5. Référence

Les personnes intéressées par une explication plus détaillée, mais en anglais, du calcul de CRC pourront lire [A Painless Guide to CRC Error Detection Algorithms](#) de Ross Williams.

3. Mise en œuvre Java

La première partie de la mise en œuvre Java de cette étape consiste à écrire un certain nombre d'enregistrements extrêmement simples représentant les différents types de chaînes contraintes décrits à la §2.3 : `IcaoAddress` pour les adresses OACI, `AircraftRegistration` pour les immatriculations, etc.

On peut légitimement se demander s'il n'aurait pas été plus simple d'utiliser le type `String` pour représenter les valeurs correspondantes, ou de n'avoir qu'une seule classe représentant les chaînes contraintes.

Cela aurait effectivement été plus simple, mais l'avantage d'avoir un enregistrement par type de chaîne contrainte est que ces enregistrements ont tous un type Java différent, et il est donc impossible de confondre les valeurs correspondantes. Par exemple, si une méthode prend en argument une valeur de type `IcaoAddress`, elle a alors la garantie que cette valeur est soit nulle, soit une adresse OACI valide. Cette garantie n'aurait pas été offerte si la méthode prenait simplement une chaîne de type `String`.

3.1. Enregistrement `IcaoAddress`

L'enregistrement `IcaoAddress`, du sous-paquetage `aircraft`, public, représente une adresse OACI. Il ne possède qu'un seul attribut :

- `String string`, la chaîne contenant la représentation textuelle de l'adresse OACI.

Le constructeur compact de cet enregistrement valide la chaîne qui lui est passée et lève

`IllegalArgumentException` si elle ne représente pas une adresse OACI valide. Cette validation est faite au moyen de l'expression régulière donnée à la fin de la §2.3.

3.1.1. Conseils de programmation

Pour vérifier qu'une chaîne est conforme à une expression régulière, on combine plusieurs méthodes des classes `Pattern` et `Matcher` de la bibliothèque Java.

Par exemple, l'extrait de code suivant initialise la variable `num2` avec une expression régulière qui décrit les nombres binaires, puis l'utilise pour déterminer si trois chaînes successives lui correspondent.

```
Pattern num2 = Pattern.compile("[01]+");
System.out.println(num2.matcher("").matches()); // false
System.out.println(num2.matcher("2023").matches()); // false
System.out.println(num2.matcher("0111").matches()); // true
```

La méthode `compile` prend en argument une chaîne de caractères représentant une expression régulière, la valide puis la transforme en un objet de type `Pattern`, qui peut ensuite être utilisé pour les tests. Pour tester si une chaîne correspond à l'expression régulière, on la passe à la méthode `matcher` de `Pattern`, puis appelle la méthode `matches` de l'objet retourné, qui est de type `Matcher`. Il n'est pas nécessaire que vous compreniez exactement à quoi sert la classe `Matcher`, il vous suffit de l'utiliser comme ci-dessus.

Notez que la méthode `compile` effectue un certain travail qu'il est préférable de ne faire qu'une fois au démarrage du programme. Faites donc en sorte que ce soit le cas, en stockant l'objet `Pattern` correspondant aux adresses OACI dans un attribut statique de l'enregistrement `IcaoAddress`.

3.2. Autres chaînes contraintes

Les enregistrement représentant les autres types de chaînes contraintes sont très similaires à `IcaoAddress` et ne sont donc pas décrits en détail. Ils sont résumés dans la table suivante – `IcaoAddress` inclus –, la dernière colonne spécifiant si oui ou non le constructeur compact doit accepter une chaîne vide.

Nom	Contenu	Vide ?
<code>IcaoAddress</code>	Adresse OACI	non
<code>AircraftRegistration</code>	Immatriculation	non
<code>AircraftTypeDesignator</code>	Indicateur de type	oui
<code>AircraftDescription</code>	Description	oui

Tous ces enregistrements appartiennent au sous-paquetage `aircraft`.

3.2.1. Conseils de programmation

Étant donné que tous les enregistrement susmentionnés sont très similaires à `IcaoAddress`, nous vous recommandons de les créer en copiant `IcaoAddress` puis en modifiant ce qui doit l'être. Pour cela, ouvrez `IcaoAddress` dans IntelliJ, puis

choisissez l'entrée *Copy class...* du menu *Refactor*.

3.3. Enregistrement `CallSign`

L'enregistrement `CallSign` du sous-paquetage `adsb (!)`, public, représente ce que l'on nomme l'**indicatif** (*call sign*) d'un aéronef.

Il s'agit d'un autre type de chaîne contrainte, qui n'a toutefois pas été décrit à la §2.3 car il ne fait pas partie des caractéristiques fixes d'un aéronef. Nous avons néanmoins choisi d'intégrer cet enregistrement à cette étape car il est lui aussi similaire à `IcaoAddress`, et il est donc logique de le programmer en même temps.

L'indicatif peut être vide, et l'expression régulière représentant l'ensemble des indicatifs valides est `[A-Z0-9 ]{0,8}`.

3.4. Type énuméré `WakeTurbulenceCategory`

Le type énuméré `WakeTurbulenceCategory` du sous-paquetage `aircraft` représente la catégorie de turbulence de sillage d'un aéronef. Il contient quatre valeurs qui sont, dans l'ordre :

- `LIGHT`,
- `MEDIUM`,
- `HEAVY`
- `UNKNOWN`.

La dernière de ces catégories est utilisée soit lorsque la catégorie réelle est inconnue, soit lorsque l'aéronef ne produit pas de turbulence importante – p. ex. s'il s'agit d'un deltaplane ou d'un ballon.

`WakeTurbulenceCategory` offre l'unique méthode publique et statique suivante, utilisée pour convertir les valeurs textuelles de la base de données en éléments du type énuméré :

- `WakeTurbulenceCategory of(String s)`, qui retourne la catégorie de turbulence de sillage correspondant à la chaîne donnée, à savoir :
 - `LIGHT` pour la chaîne "L",
 - `MEDIUM` pour la chaîne "M",
 - `HEAVY` pour la chaîne "H",
 - `UNKNOWN` pour toutes les autres chaînes.

3.5. Enregistrement `AircraftData`

L'enregistrement `AircraftData`, du sous-paquetage `aircraft`, collecte les données fixes d'un aéronef. Il possède les attributs suivants, dont la signification devrait être évidente :

- `AircraftRegistration registration`,
- `AircraftTypeDesignator typeDesignator`,
- `String model`,
- `AircraftDescription description`,

- `WakeTurbulenceCategory wakeTurbulenceCategory`.

Le constructeur compact de `AircraftData` lève `NullPointerException` si l'un de ses arguments est nul. Pour ce faire, il appelle la méthode `requireNonNull` de `Objects` sur chacun d'entre eux.

3.6. Classe `AircraftDatabase`

La classe `AircraftDatabase` du sous-paquetage `aircraft`, publique et finale, représente la base de données micronics des aéronefs.

Attention : pour pouvoir écrire cette classe, vous devriez avoir suivi le cours sur les entrées/sorties, présenté lors de la seconde semaine du semestre. Si vous ne l'avez pas encore suivi, il peut être préférable de commencer par faire la classe `Crc24` décrite à la section suivante.

La classe `AircraftDatabase` possède un unique constructeur public :

- `AircraftDatabase(String fileName)`, qui retourne un objet représentant la base de données micronics, stockée dans le fichier de nom donné, ou lève `NullPointerException` si celui-ci est nul.

Ce constructeur ne fait rien d'autre que vérifier que son argument n'est pas nul avant de le stocker dans un attribut de la classe. En particulier, il ne lit aucune donnée de la base de données, cette tâche étant réservée à la seule méthode publique de la classe :

- `AircraftData get(IcaoAddress address)`, qui retourne les données de l'aéronef dont l'adresse OACI est celle donnée, ou `null` si aucune entrée n'existe dans la base pour cette adresse ; lève `IOException` en cas d'erreur d'entrée/sortie.

Notez que la méthode `get` doit tirer parti du fait que les fichiers de la base de données sont triés par ordre croissant d'adresse OACI, et interrompre la recherche aussi rapidement que possible lorsque l'adresse OACI recherchée ne figure pas dans le fichier.

3.6.1. Conseils de programmation

Les 256 fichiers contenant les différentes parties de la base de données se trouvant à l'intérieur d'une archive Zip, il n'est pas possible d'y accéder directement au moyen des classes vues au cours (`FileInputStream` ou `FileReader`).

Au lieu de cela, il faut utiliser la classe `ZipFile`, qui permet d'accéder au contenu d'une archive Zip. La méthode `getEntry` permet d'obtenir une entrée (c.-à-d. un fichier) de l'archive à partir de son nom, et la méthode `getInputStream` permet ensuite d'obtenir un flot d'entrée avec le contenu de cet entrée.

L'utilisation de ces méthodes est illustrée par l'extrait de programme ci-dessous, qui affiche à l'écran la totalité du contenu du fichier `14.csv` de l'archive Zip. Les variables ont été nommées au moyen d'une seule lettre par manque de place, mais il va de soi que vous devez utiliser des noms plus parlants dans votre code.

```
String d = getClass().getResource("/aircraft.zip").getFile();
d = URLDecoder.decode(d, UTF_8);
try (ZipFile z = new ZipFile(d);
    InputStream s = z.getInputStream(z.getEntry("14.csv"));
    Reader r = new InputStreamReader(s, UTF_8);
    BufferedReader b = new BufferedReader(r)) {
    String l = "";
    while ((l = b.readLine()) != null)
        System.out.println(l);
}
```

Cet extrait de code illustre également la manière correcte – mais un peu obscure – d'obtenir le nom du fichier contenant la base de données, en combinant les méthodes `getClass`, `getResource` et `getFile`.

Pour trouver la ligne correspondant à l'adresse OACI recherchée, les méthodes `startsWith` et `compareTo` de `String` pourront vous être utiles. Une fois la ligne trouvée, il est facile de la découper en colonnes au moyen de la méthode `split` à laquelle on passe le séparateur en premier argument et `-1` en second argument (!).

3.7. Classe Crc24

La classe `Crc24` du paquetage principal, publique, finale et immuable, représente un calculateur de CRC de 24 bits. Elle possède un unique attribut public et statique :

- `int GENERATOR`, qui contient les 24 bits de poids faible du générateur utilisé pour calculer le CRC24 des messages ADS-B, qui vaut FFF409_{16} .

Notez bien que le générateur « réel » fait 25 bits, mais que comme nous l'avons expliqué, le stockage du bit de poids le plus fort n'est pas nécessaire, car celui-ci vaut toujours 1.

En plus de cet attribut, la classe `Crc24` offre un constructeur public :

- `Crc24(int generator)`, qui retourne un calculateur de CRC24 utilisant le générateur dont les 24 bits de poids faible sont ceux de `generator`.

En dehors de ce constructeur, la classe `Crc24` offre une seule méthode publique :

- `int crc(byte[] bytes)`, qui retourne le CRC24 du tableau donné.

Notez que le calcul du CRC24 doit impérativement être fait avec l'algorithme le plus optimisé, travaillant octet par octet et présenté à la §2.4.4.

3.7.1. Conseils de programmation

1. Organisation

L'algorithme travaillant octet par octet utilise une table dont la construction est faite par l'algorithme travaillant bit par bit. Nous vous conseillons donc de vous organiser ainsi pour programmer la classe `Crc24` :

1. écrivez l'algorithme bit par bit dans une méthode privée et statique nommée

- p. ex. `crc_bitwise`, prenant en arguments le générateur et le tableau d'octets dont le CRC24 doit être calculé, et retournant celui-ci,
2. écrivez ensuite une version basique de la méthode `crc`, qui ne fait rien d'autre qu'un appel à `crc_bitwise`,
3. écrivez un ou plusieurs tests unitaires pour vérifier que cette première mise en œuvre fonctionne correctement (voir plus bas),
4. ajoutez une méthode privée et statique à votre classe, nommée p. ex. `buildTable`, construisant la table de 256 entrées correspondant à un générateur, en utilisant la méthode `crc_bitwise`,
5. modifiez le constructeur pour qu'il construise la table et la stocke dans un attribut, et la mise en œuvre de `crc` pour qu'elle utilise l'algorithme optimisé.

Si vous êtes à court de temps, vous pouvez même rendre votre projet avec la version basique de l'algorithme uniquement, qui suffit pour passer les tests – à condition qu'elle soit correcte, bien entendu. Ensuite, **avant le rendu intermédiaire**, écrivez la version optimisée de l'algorithme.

2. Augmentation du message

Les algorithmes présentés aux §2.4.3 et §2.4.4 travaillent sur le message augmenté, c.-à-d. auquel 24 bits valant 0 ont été ajoutés. En pratique, plutôt que de réellement augmenter le message, ce qui impliquerait de copier le tableau, écrivez deux boucles successives : la première traitant les bits (ou octets) du message, la seconde traitant les 24 bits (ou 3 octets) ajoutés. Ces derniers valant tous 0, la seconde boucle est une version simplifiée de la première.

3. Tests

Pour tester votre classe `Crc24`, vous pouvez vérifier qu'elle fonctionne correctement sur les valeurs suivantes, qui ne sont rien d'autre que des représentations hexadécimales de messages ADS-B valides :

- 8D392AE499107FB5C00439035DB8,
- 8D4D2286EA428867291C08EE2EC6,
- 8D3950C69914B232880436BC63D3,
- 8D4B17E399893E15C09C219FC014,
- 8D4B18F4231445F2DB63A0DEEB82,
- 8D495293F82300020049B8111203.

Les trois derniers octets, mis en évidence en rose, constituent le CRC24 du message, formé des octets précédents. Vous pouvez donc vérifier que votre méthode `crc` retourne 0 lorsqu'on lui donne le message accompagné du CRC, ou les 3 derniers octets lorsqu'on lui donne le message seul. Par exemple, un test utilisant le premier de ces messages pourrait ressembler à :

```
Crc24 crc24 = new Crc24(Crc24.GENERATOR);
String mS = "8D392AE499107FB5C00439";
String cS = "035DB8";
int c = Integer.parseInt(cS, 16); // == 0x035DB8
```

```
byte[] mAndC = HexFormat.of().parseHex(mS + cS);  
assertEquals(0, crc24.crc(mAndC));  
  
byte[] mOnly = HexFormat.of().parseHex(mS);  
assertEquals(c, crc24.crc(mOnly));
```

3.8. Tests

À partir de cette étape, nous ne vous fournissons plus de tests unitaires, et il vous faut donc les écrire vous-même.

Nous vous fournissons néanmoins [un fichier de vérification de signatures](#), nommé `SignatureChecks_2.java`, à importer dans votre projet dans le même dossier que celui contenant le fichier `SignatureChecks_1.java`. La classe qu'il contient fait référence à la totalité des classes et méthodes de cette étape, ce qui vous permet de vérifier que leurs noms et types sont corrects. Cela est capital, car la moindre faute à ce niveau empêcherait l'exécution de nos tests unitaires.

Nous vous fournirons de tels fichiers pour toutes les étapes jusqu'à la sixième (inclusive), et il vous faudra penser à vérifier systématiquement qu'aucune erreur n'est signalée à leur sujet. Faute de cela, votre rendu pourrait se voir refusé par notre système.

4. Résumé

Pour cette étape, vous devez :

- écrire les classes et enregistrements `IcaoAddress`, `AircraftRegistration`, `AircraftTypeDesignator`, `AircraftDescription`, `CallSign`, `WakeTurbulenceCategory`, `AircraftData`, `AircraftDatabase` et `Crc24` selon les indications ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies,
- rendre votre code au plus tard le **3 mars 2023 à 18h00**, au moyen du programme `Submit.java` fourni et des jetons disponibles sur votre [page privée](#).

Ce rendu est un rendu testé, auquel 18 points sont attribués, au prorata des tests unitaires passés avec succès.

N'attendez surtout pas le dernier moment pour effectuer votre rendu, car vous n'êtes pas à l'abri d'imprévus. **Souvenez-vous qu'aucun retard, aussi insignifiant soit-il, ne sera toléré !**

Notes de bas de page

¹ Une citation célèbre de [Jamie Zawinski](#) résume bien cet état de fait : « *Some people, when confronted with a problem, think “I know, I'll use regular expressions.” Now they have two problems.* »

² En réalité, le calcul de CRC est basé sur une division de polynômes, mais il n'est pas nécessaire de

savoir cela pour comprendre la technique.